Université Joseph Ki-Zerbo · IFOAD

Master 2 Data Science

UE Bases de données NoSQL

TP final FasoPay

Audit, conception et démonstration
d'un portefeuille de paiement mobile sur MongoDB

Groupe 07

YAMEOGO Joseph

ZONGO Abdrafith

Enseignant : M. Gael BARRY

Année universitaire 2026-2027 · Novembre 2026

Table des matières

1 Contexte et problématique **3**

2 Partie 0 · Dimensionnement **4**

3 Partie 1 · Modélisation (ADR-001) **4**

4 Partie 2 · Enquête : E1, E2, E3, E4 **6**

5 Partie 3 · Indexation **8**

6 Partie 4 · Transactions **8**

7 Partie 5 · Réplication (ADR-002) **10**

8 Partie 6 · Sharding **11**

9 Partie 7 · Architecture proportionnée (ADR-003) **12**

10 Synthèse CTO : 5 priorités pour les 12 prochains mois **16**

11 Bonus B2 · Relais Outbox par Change Stream **17**

12 Conclusion et limites du POC **17**

Annexe A · Feuille de dimensionnement **19**

Annexe B · Journal de montage **23**

Annexe C · Code **28**

Annexe D · Preuves **29**

Annexe E · Déclaration d'usage de l'IA **29**

1 Contexte et problématique

FasoPay, la filiale de paiement mobile de Faso Hub, vise 3 millions de wallets et 400 millions de transactions par an à 18 mois, avec un pic de 1 200 transactions par seconde, une disponibilité de 99,95 % et **aucune perte d'argent acceptable** (RPO de 0). Son CTO nous confie quatre demandes, qui structurent ce rapport :

Demande du CTO	Notre réponse	Où
(1) un modèle de données solide	ledger en partie double, validateurs qui rejettent, solde traité comme un cache	Parties 0 et 1
(2) retrouver les soldes incohérents	6 wallets trouvés et chiffrés, plus le découpage, le CA et les rejeux	Partie 2
(3) des virements qui ne doublent ni ne se perdent	`transfer()` : idempotence, abort prouvé, concurrence, pannes réelles	Parties 3, 4, 5
(4) une architecture sans « usine à gaz »	cible PME chiffrée, chaque composant écarté avec son seuil de retour	Parties 6, 7

Notre principe de travail. Nous ne prétendons rien que nous n'ayons mesuré nous-mêmes. Chaque chiffre de ce rapport renvoie à un fichier de `preuves/` (commande exacte, date `date -u`, sortie brute), produit par une commande de l'annexe B. Quand une mesure contredit notre architecture, nous la montrons au lieu de la cacher (Partie 0).

Notre jeu de données (groupe 07)

2 000 clients, 150 marchands, 60 000 transactions sur 181 jours à partir du 1er janvier 2026. Notre vérification D3, écrite par nous, retrouve chaque chiffre du `fingerprint.json` (`preuves/00-fingerprint.txt`) :

Grandeur	Attendu	Obtenu
customers / wallets / merchants	2 000 / 2 000 / 150	2 000 / 2 000 / 150
transactions / ledger_entries	60 000 / 141 939	60 000 / 141 939
somme des `amount` (toutes / `COMPLETED`)	2 018 172 375 / 1 946 303 325	identiques
somme des `balance` des wallets	832 251 836	832 251 836
empreintes SHA-256 des 5 fichiers NDJSON	5	5 conformes

Les types ont été contrôlés après l'import : montants en `int`, dates en `date`, seul `feePct` en `double` (`preuves/00-types.txt`). Les collections de traitement `outbox` et `fraud_alerts` ont été créées vides (D4).

Notre environnement

Windows, Docker 29.8.0, MongoDB 8.0.34 (image `mongo:8.0`) et mongosh 2.14.0, avec environ 8,17 Go de mémoire visibles par Docker. Nous avons monté nous-mêmes quatre environnements, tous décrits commande par commande en annexe B : le serveur du TP `mongo-rs1` (Replica Set `rs0` à un nœud, Parties 0 à 4), le Replica Set `rs-fasopay` à trois nœuds (Partie 5), un cluster shardé à trois shards (Partie 6) et un serveur de test avec authentification (Partie 7).

Pourquoi un Replica Set, même à un seul nœud ? Les transactions multi-documents, indispensables à un virement (Partie 4), s'appuient sur l'oplog, le journal des opérations que tient un Replica

Set. Un serveur MongoDB isolé les refuse. Le mode Replica Set est donc obligatoire dès la Partie 0, même avec un seul membre ; à un nœud, la majorité vaut 1 et `w: "majority"` équivaut à `w: 1`, ce qui nous a conduits à rejouer nos tests sur trois nœuds (Partie 5). État vérifié : `set: rs0`, `myState: 1` (PRIMARY), un membre (`preuves/00-rs-status-1-noeud.txt`).

2 Partie 0 · Dimensionnement

Nous avons mesuré nos documents avec `$bsonSize` sur toute la collection, et la taille des données et des index avec `$collStats` (`preuves/00-dimensionnement.txt` ; feuille complète en annexe A). Une transaction pèse 253 octets, une ligne de ledger 130, et une transaction métier écrit 2,37 lignes de ledger : 561 octets par transaction métier.

Grandeur à 18 mois	**Calcul**	**Résultat**
Données par an	400 M × 561,02 octets	224,4 Go
Avec index	facteur d'index mesuré ×1,37 (après les index de la Partie 3)	307,9 Go
Écritures au pic	1 200 tx/s × 6 documents (ADR-001)	7 200 documents/s
Working set	30 jours de transactions (18,4 Go) + leurs index (6,9 Go) + 3 M wallets et leur index (0,6 Go)	**25,9 Go**
RAM par nœud	cache ≈ 50 % × (RAM − 1 Go) ≥ 25,9 Go, donc RAM ≥ 52,8 Go	**64 Go**
Disque	198,5 Go par an après compression (0,51 mesurée), sur 1,4 To utilisables	7,1 ans
Facteur de pic	pic ÷ moyenne = 1 200 ÷ 12,7	×95

Le facteur de pic de 95 est la leçon de cette feuille : **nous dimensionnons sur le pic, pas sur la moyenne**. Ces chiffres pointent vers une architecture proportionnée : un seul Replica Set, sans sharding, sans Spark, sans Redis et sans Kafka au démarrage. Chaque composant écarté a un seuil de retour chiffré (annexe A, section 3), par exemple le sharding entre 490 et 600 M transactions par an sur des nœuds de 64 Go.

Deux chiffres qui contredisent notre verdict, et ce que nous en faisons.

— **La taille des index.** Nos index pèsent 27 octets par entrée, parce qu'un index de 60 000 documents reste compact. À grande échelle, on compte plutôt 40 à 60 octets (chapitre 11). À 60 octets, le working set monterait à 34,3 Go et **64 Go ne suffiraient plus**. Nous gardons 64 Go, avec un seuil de surveillance : au-delà de 45 octets par entrée, nous passons à des nœuds de 128 Go (une montée en gamme, pas un sharding).

— **Le débit d'écriture.** Sur notre portable, `transfer()` atteint 330 virements/s avec 4 clients (Partie 4). Pour que le pic de 1 200 n'occupe que la moitié du primary, il en faudrait 2 400. La mesure ne vaut pas pour un serveur, mais elle nous impose une décision : **un test de charge sur le matériel cible avant la mise en production**.

À retenir. un Replica Set de nœuds de 64 Go tient la cible à 18 mois en volume et en mémoire ; le débit du primary au pic reste à prouver sur le matériel cible, et nous le disons.

3 Partie 1 · Modélisation (ADR-001)

3.1 Un schéma qui rejette ce qui est faux

Six collections portent un validateur `$jsonSchema` en `validationLevel: "strict"` et `validationAction: "error"` : `customers`, `wallets`, `merchants`, `transactions`, `ledger_entries` et `outbox`. Les montants sont des entiers (`int` ou `long`) : un franc CFA ne se divise pas, et un `double` introduirait des arrondis. Le validateur de `ledger_entries` utilise `oneOf` pour accepter les deux formes de ligne : une jambe

de wallet (`walletId` et `balanceAfter` obligatoires) ou une jambe hors wallet (`walletId: null` et un compte `FEES`, `FLOAT`, `BILLER` ou `MERCHANT:MâÃę`).

Preuve (`preuves/01-validator-rejet.txt`) : nos 206 089 documents passent tous (0 invalide), et un document invalide est **rejeté dans chaque collection** avec le code 121 et son détail `errInfo` (par exemple `balance: 12.5` ou `kycLevel: 'trois'`).

3.2 Embarquer ou référencer : relation par relation

Relation	Choix	Justification mesurée
client ↔ wallet	référence	1 wallet par client ; le wallet change à chaque virement, le client presque jamais
wallet ↔ transactions	référence	une transaction concerne deux wallets ; jusqu'à 319 transactions par wallet
transaction ↔ ledger	collection séparée	2 à 6 lignes par transaction, relues par wallet (historique) et pas seulement par transaction
transaction ↔ tentatives	embarqué	les rejeux se lisent avec leur transaction ; seules 21 en ont
ledger ↔ solde	copie (`balanceAfter`)	l'historique d'un wallet se lit sans recalcul (Computed pattern)

3.3 Quatre access patterns, par fréquence × criticité

	Access pattern	Au pic	Latence cible	Cohérence
AP1	exécuter un virement (avec la clé d'idempotence)	1 200/s	p95 < 300 ms	forte
AP2	lire son solde (USSD, application)	milliers/s	p95 < 100 ms	forte (primary)
AP3	historique d'un wallet, trié, paginé	centaines/s	p95 < 150 ms	retard toléré
AP4	contrôles de nuit : rapprochement (E1), plafond (E2)	1 par nuit	minutes	photo figée

AP1 est le plus critique (c'est de l'argent, au pic), AP2 le plus fréquent. AP4 est rare, mais c'est lui qui détecte les écarts dont se plaint le CTO.

3.4 L'anti-pattern évité : le ledger embarqué dans le wallet

Mettre les lignes de ledger dans un tableau du wallet semble naturel. Mesuré sur nos données : 35,94 lignes par wallet en moyenne et 390 au maximum, soit 4 831 octets en moyenne et 52 346 au maximum, après six mois seulement. Le wallet, document le plus modifié de la base, grossirait à chaque virement et serait réécrit en entier, jusqu'à la limite de 16 Mo (environ 125 000 lignes). Nous gardons donc le ledger séparé, relié par `walletId`.

3.5 Un grand livre en partie double

Notre modèle applique les règles d'un grand livre, toutes vérifiées sur nos données : **montants entiers** en FCFA ; **partie double** (débits = crédits = 1 955 479 314 FCFA, 0 transaction déséquilibrée sur 56 484) ; **ajout seul** (les 280 transactions `REVERSED` gardent leurs lignes d'origine et reçoivent des lignes inverses, `preuves/01-mesures-modelisation.txt`) ; **le solde est un cache** du ledger, qui doit changer dans la même transaction que lui, sinon il diverge (ce que mesure E1) ; **idempotence** par un index unique (Partie 3).

ADR-001 (décision). Un virement écrit **6 documents dans une seule transaction multi-documents** : la transaction, deux lignes de ledger, deux soldes et un événement `outbox`. Rejetés : la transaction seule (aucun solde auditable), le solde recalculé à chaque lecture (35,94 lignes en moyenne,

toujours plus), le solde sans ledger (aucune trace pour un solde contesté). Coût accepté : 7 200 écritures/s au pic. ADR complet dans `adr/ADR-001.md`.

4 Partie 2 · Enquête : E1, E2, E3, E4

Chaque pipeline commence par un `$match` qui réduit les documents le plus tôt possible ; l'ordre des étapes est commenté dans `code/02-enquete.js`. Toutes les réponses se rattachent au `fingerprint.json` (`preuves/02-enquete.txt`).

4.1 E1 : les soldes qui ne collent pas au ledger

Pour chaque wallet, nous additionnons ses lignes de ledger (crédit +, débit −) et comparons à `wallets.balance`. Le `$match` écarte d'abord les 70 054 lignes sans wallet, et le `$group` précède le `$lookup` : 2 000 jointures au lieu de 71 885.

Wallet	Solde stocké	Solde du ledger	Écart	Lignes
W001124	249 025	278 025	−29 000	7
W001372	557 152	585 152	−28 000	33
W000967	389 447	415 447	−26 000	28
W001796	949 293	960 793	−11 500	49
W000668	478 654	487 654	−9 000	31
W000769	75 692	41 192	+34 500	86

6 wallets sur 2 000 sont incohérents. Les écarts se compensent en partie : l'écart net (−69 000 FCFA) sous-estime le problème, nous retenons l'**écart absolu de 138 000 FCFA**. **Rattachement :** le ledger signé des wallets totalise 832 320 836, les soldes 832 251 836 (fingerprint) ; la différence, 69 000, est exactement l'écart net : rien d'autre ne diverge. Pour ces 6 wallets, le dernier `balanceAfter` du ledger égale la somme du ledger : **c'est le solde stocké qui a divergé, pas le ledger**. Le correctif passe par des écritures d'ajustement dans le ledger, pas par une modification directe du solde.

4.2 E2 : les clients qui découpent leurs transferts

Notre critère, explicite : un wallet est suspect un jour donné s'il émet au moins 3 sorties `COMPLETED` (`P2P` ou `CASH_OUT`) comprises chacune entre 90 % et 100 % de son plafond `dailyLimit`. Les sorties vers une personne ou en espèces sont les transferts ; la bande 90–100 % décrit un montant choisi pour rester juste sous la limite ; le seuil de 3 évite de signaler un cas isolé.

Résultat : 5 wallets, 23 couples wallet-jour, 121 transactions. Le critère est solide : aucune sortie ne dépasse le plafond à elle seule (0 sur 46 495), aucun autre wallet n'a de transaction dans la bande, et avec un seuil de 50 % ou 70 % nous retrouvons les mêmes 5 wallets. **Le contournement est un problème de cumul, pas de montant.**

Cas détaillé : W000758, le 30 janvier 2026 (plafond 200 000 FCFA). Sept sorties en moins de 3 heures, espacées de 21 à 41 minutes :

Heure (UTC)	Type	Destinataire	Montant (FCFA)
08:25	CASH_OUT	—	191 375
08:54	P2P	W000951	186 050
09:20	CASH_OUT	—	189 575
09:41	P2P	W001312	181 850
10:17	P2P	W001312	191 975
10:40	P2P	W000951	191 475
11:21	CASH_OUT	—	188 025
Total de la matinée			**1 320 325**

Soit 6,6 fois le plafond en une matinée, vers deux bénéficiaires seulement. Un indice n'est pas une preuve de fraude : ces 5 wallets sont la liste de départ de la conformité, et le contrôle cumulatif doit devenir automatique (alerte dans `fraud_alerts`, Partie 7).

4.3 E3 : les marchands qui concentrent le chiffre d'affaires

Notre définition du CA : la somme des `amount` des `MERCHANT_PAY` au statut `COMPLETED` ; les échecs n'ont rien encaissé, les remboursés sont rendus, et la commission (`fee`) appartient à FasoPay. Compter tous les statuts gonflerait le CA de 10,9 % : la définition change le chiffre, d'où la nécessité de la fixer. CA marchand : 229 050 525 FCFA, sur 16 603 paiements.

Part des marchands	10 % (15)	**20 % (30)**	30 %	50 %	70 %	100 %
Part cumulée du CA	72,55 %	**81,96 %**	87,62 %	94,28 %	97,94 %	100 %

Oui, le top 20 % fait plus de 80 % (81,96 %). Trois marchands font à eux seuls 54,51 % du CA : M00130 (École Paspanga, 27,21 %), M00044 (Faso Market Central, 18,09 %) et M00062 (9,21 %). M00044 domine par le **nombre** de paiements (4 162, soit 25,07 % du volume marchand) : c'est le « marchand-hotspot » que nous retrouverons au sharding (Partie 6).

4.4 E4 : les clés rejouées et l'incident

21 clés ont été rejouées : 86 tentatives, dont 21 traitées (exactement une par clé) et 65 rejeux, arrivés de 6,9 à 138,2 secondes après la première tentative. **Aucun écart de montant ni de statut** : chaque clé n'a qu'un document, les 21 transactions sont `COMPLETED` avec le nombre habituel de lignes de ledger, et notre contrôle des doubles paiements marchands ne trouve rien. Pour être sûrs qu'un contrôle qui renvoie 0 sait trouver un cas réel, nous l'avons testé sur une copie jetable avec un faux doublon ajouté exprès : il le trouve.

L'incident : le 10 avril 2026, de 14:09 à 15:36 UTC (87 minutes), 10 clés différentes sont rejouées, avec 36 rejeux, alors que les 11 autres jours n'en comptent qu'une chacun. Cette concentration est compatible avec la coupure réseau décrite par le CTO. **Sans idempotence, les 65 rejeux auraient débité 633 025 FCFA en trop.**

À retenir. l'argent n'a pas été doublé dans nos données, mais 6 soldes ont divergé du ledger : le risque principal est un chemin d'écriture qui modifie le solde hors de la transaction qui écrit le ledger.

5 Partie 3 · Indexation

Nos requêtes critiques viennent de l'enquête : **Q1**, les sorties du jour d'un wallet (le contrôle du plafond de E2) ; **Q2**, la dernière ligne de ledger d'un wallet (le rapprochement de E1) ; **Q3**, la recherche par `idempotencyKey`. Chaque index suit la règle ESR (Égalité, Tri, Plage). Les temps sont des médianes de

3 exécutions ; le « avant » est forcé par `hint({ $natural: 1 })` (`preuves/03-explain.txt`, JSON bruts).

	Index (ESR)	Plan avant → après	Documents lus	Médiane
Q1	`{ fromWalletId, status, createdAt, amount }` : E, E, R, puis `amount` pour couvrir	`COLLSCAN` → `PROJECTION_COVERED > IXSCAN`	60 000 → **0**	19 → 0 ms
Q2	`{ walletId: 1, createdAt: -1 }` : E, S	`SORT > COLLSCAN` → `LIMIT > FETCH > IXSCAN`	141 939 → 1	43 → 0 ms
Q3	`{ idempotencyKey: 1 }`, **unique**	`COLLSCAN` → `EXPRESS_IXSCAN`	60 000 → 1	18 → 0 ms

Q1 est un **index couvrant** : la requête est servie par l'index seul, sans lire aucun document. Pour Q2, l'ordre de l'index correspond au tri demandé : l'étape `SORT` disparaît, et le même index sert l'historique complet du wallet. Pour Q3, l'index unique est la base de l'idempotence : une insertion avec une clé existante échoue avec le code 11000.

Ce que coûtent les index. +2,54 Mo sur `transactions` (16,7 % de ses données) et +2,16 Mo sur `ledger_entries` (11,7 %) ; à chaque virement, 4 entrées d'index de plus. **Un cas où l'index n'aide pas :** la même recherche que Q1 sans `fromWalletId` ne peut pas utiliser l'index (préfixe gauche) : `COLLSCAN`, 104 documents lus par document renvoyé, au-dessus du seuil de 100 qui signale un index manquant (chapitre 12). Nous ne l'indexons pas : ce n'est pas un access pattern retenu.

Un index refusé, mais mesuré. L'index `{ type, status, merchantId, amount }` accélère E3 de 23 à 9 ms. Nous l'avons créé le temps de la mesure, puis supprimé : il coûterait une entrée d'index à chacune des 400 millions de transactions annuelles (environ 4 Go par an) pour gagner 14 ms sur un rapport rare. Le bon outil est un read model alimenté par `$merge` et lu sur un secondary (Partie 7).

6 Partie 4 · Transactions

6.1 Comment fonctionne transfer()

`code/transfer.js` (annexe C) respecte le contrat de l'énoncé. Tous nos tests tournent dans une base à part, `fasopay_test`, recréée à chaque lancement avec nos validateurs : `fasopay_tp` n'est jamais modifiée.

1. **Contrôles sans base :** montant entier strictement positif, stocké en `int` ou `long`.
2. **Rejeu :** si la clé existe déjà, la fonction renvoie le même `txId`, sans rien écrire.
3. **Transaction** en `w: "majority"` et lecture `snapshot` : d'abord le document `transactions` (l'index unique empêche deux virements pour une clé), puis le **débit conditionnel** (wallet `ACTIVE` et solde suffisant, vérifiés dans la même opération), le crédit, les deux lignes de ledger et l'événement `outbox`.
4. **Erreurs :** tout est annulé ; une clé en double (code 11000) devient un rejeu ; `TransientTransactionError` rejoue la transaction ; `UnknownTransactionCommitResult` redemande la validation.

6.2 Ce que nos tests démontrent

36 vérifications OK sur 36 (`preuves/04-transaction.txt`), puis les mêmes sur le Replica Set à trois nœuds, où la majorité attend vraiment un second nœud (`preuves/04-transaction-rs3.txt`).

Test	Résultat
Virement réussi	1 transaction, 2 lignes de ledger, 1 outbox ; soldes et `balanceAfter` justes ; montants en `int`
3 rejeux	1 seul document, un seul débit (2 000 et non 8 000), même `txId` pour les 3 rejeux
10 refus	la bonne erreur à chaque fois (montant décimal, nul, négatif, texte, soi-même, wallet absent ou gelé, solde insuffisant), rien d'écrit
Réponse perdue	le client réessaie avec la même clé : rejeu reconnu, un seul débit
Abort prouvé	voir ci-dessous

L'abort, prouvé et pas seulement affirmé. Avec `failAfterFirstWrite`, nous montrons trois choses : juste après la première écriture, le document est lu **dans** la transaction alors qu'une lecture **hors** de la transaction renvoie `null` ; le profileur du serveur a bien enregistré l'insertion, et le compteur d'annulations du serveur passe de 5 à 6 ; après `CRASH_SIMULATED`, le document a disparu et les soldes sont intacts.

6.3 Concurrence, et ce qui arrive sans transaction

Deux clients partent à la même milliseconde et envoient chacun 25 virements de 100 FCFA depuis un wallet de 3 000 FCFA. **10 vérifications OK sur 10** (`preuves/04-concurrence.txt`) : exactement **30 virements acceptés** (19 + 11), 20 refus, solde final 0 et jamais négatif, monnaie conservée, ledger = 60 = 2 × 30, outbox = 30 ; MongoDB a signalé 6 conflits, tous rejoués par `transfer()`.

Pour montrer ce que nos protections évitent, nous avons lancé la version qu'on écrirait spontanément : lire le solde, vérifier, réécrire, sans transaction ni condition (`preuves/04-lost-update.txt`). **Résultat : 50 débits acceptés au lieu de 30, 5 000 FCFA promis pour 2 500 FCFA réellement retirés.** C'est le « lost update » : deux clients lisent le même ancien solde, et un débit sur deux est perdu. C'est une des causes possibles des 6 soldes divergents de E1.

Débit mesuré (`preuves/04-debit.txt`, sur une copie des 2 000 vrais wallets) : 125 virements/s avec un client, **330/s avec 4 clients**, p95 à 17 ms, somme des soldes inchangée. C'est un ordre de grandeur de portable, pas la capacité d'un serveur (Partie 0).

6.4 Pourquoi la transaction ne suffit pas quand la réponse se perd

La transaction garantit le « tout ou rien » des 6 écritures, mais ne dit rien au client qui n'a pas reçu la réponse : il ne sait pas si la coupure est tombée avant ou après la validation, et réessaie. Sans protection, ce réessai crée un second virement : c'est la « rafale de paiements en double » du CTO. Ce qui la rend impossible : **la clé d'idempotence** et son index unique (le réessai reçoit la réponse du premier virement), complétés par les écritures rejouables (`retryWrites`) et le traitement des deux étiquettes d'erreur. Aucun ne protège un réessai avec une *nouvelle* clé : le client doit fixer sa clé avant le premier envoi.

7 Partie 5 · Réplication (ADR-002)

7.1 Le Replica Set à trois nœuds

Nous avons monté `rs-fasopay` : trois nœuds (`fasopay-noeud1` à `3`), chacun dans son conteneur et avec son volume, sur le réseau Docker `reseau-fasopay`. `rs.status()` montre un PRIMARY et deux SECONDARY au même optime, 3 membres votants et une **majorité d'écriture de 2** (`preuves/05-rs-status.txt`) : une écriture `w: "majority"` n'est acquittée qu'une fois présente sur deux nœuds.

Résolution des noms (`ENOTFOUND`). Les membres s'annoncent par leur nom sur le réseau Docker, que Windows ne sait pas résoudre. Nous avons choisi de lancer le client dans un conteneur à part,

`fasopay-client`, sur le même réseau : il résout les noms par le DNS interne de Docker, il ne tourne sur aucun nœud et survit donc à la panne du primary, et la machine Windows n'est pas modifiée. L'alternative, déclarer les trois noms dans le fichier `hosts` de Windows et publier un port par nœud, demande des droits administrateur.

7.2 Trois pannes réelles, mesurées

Un client écrit en continu (une écriture toutes les 100 ms) et note chaque résultat ; au bout de 20 s, nous provoquons la panne. Le journal brut de chaque écriture est dans la preuve.

	Panne du primary (`docker kill`), majority	**Même panne, client réglé à 1 s**	**Coupure réseau du primary, w: 1**
Élection du nouveau primary	11,32 s	10,31 s	10,98 s
Plus longue interruption	21,17 s	**10,62 s**	—
Écritures acquittées puis perdues	**0** (627 acquittées)	**0**	**97**
Preuve (`preuves/`)	`05-failover.txt`	`05-failover-hb1.txt`	`05-failover-w1.txt`

Pourquoi 21 s d'interruption pour 11 s d'élection ? Nous avons formulé une hypothèse et l'avons testée : par défaut, le pilote ne vérifie l'état des nœuds que toutes les 10 s (`heartbeatFrequencyMS`), et continue donc à chercher l'ancien primary après l'élection. Avec une vérification par seconde, la première écriture passe 0,44 s après l'élection au lieu de 9,99 s, et **l'interruption est divisée par deux**. Une élection d'environ 10 s par mois consomme 0,8 % du budget d'indisponibilité de 99,95 % (21,6 minutes par mois).

Ce que `w: 1` fait perdre, et où MongoDB le range. Coupé du réseau, l'ancien primary se croit encore primary pendant 10 s et **acquitte 97 écritures** que personne ne recopie. Rebranché, il découvre le nouveau primary et les annule (rollback). Le client a reçu « acquitté » pour chacune, et aucune n'est dans la base. MongoDB les range dans `/data/db/rollback/`, où nous retrouvons les 97 documents avec `bsondump` (`preuves/05-rollback.txt`) ; rien ne les réintègre automatiquement. En `w: "majority"`, ces écritures n'auraient jamais été acquittées : le client aurait su qu'elles n'étaient pas garanties et les aurait rejouées avec sa clé d'idempotence.

7.3 ADR-002 : quel niveau de garantie pour quel usage

Usage	**Écriture et lecture**	**Raison mesurée**
Argent (virements, ledger, outbox, solde qui décide)	`w: "majority"`, `j: true`, `wtimeout` ; lecture sur le primary en `readConcern: "majority"`, `snapshot` dans les transactions	0 écriture acquittée perdue en majority, 97 en `w: 1`
Tableau de bord	read model recalculable ; lecture `secondaryPreferred` avec `maxStalenessSeconds: 90`	ne pas consommer le primary, notre point le plus fragile

Le client est réglé avec `retryWrites=true` et `heartbeatFrequencyMS=1000`. Rejetés : `w: 1` pour l'argent (97 pertes en une coupure) et `w: 3`, qui bloquerait toutes les écritures dès qu'un seul nœud tombe. Coût accepté : une écriture d'argent attend un deuxième nœud (6 à 7 ms en médiane sur notre machine). ADR complet dans `adr/ADR-002.md`.

8 Partie 6 · Sharding

Nous avons monté un cluster complet, pièce par pièce : un config server, trois shards (chacun un Replica Set) et un routeur `mongos` ; `config.shards` liste les trois shards et le balancer est actif (`preuves/06-cluster.txt`). Pour chaque clé candidate, la collection est shardée **avant** l'import, les 60 000 transactions sont importées **à travers le mongos**, puis nous insérons 3 000 transactions datées du jour pour voir où tombe le trafic de demain (`preuves/06-cle-*.txt`).

Clé	Répartition des documents	Trafic du jour (shard 1 / 2 / 3)	Historique d'un wallet
`{ createdAt: 1 }`	1 chunk sur un shard, corrigé à la main	0 / 0 / **100 %** : hotspot	3 shards
`{ fromWalletId: 1 }`	1 chunk, corrigé à la main	33 / 33 / 34 %	1 shard
`{ fromWalletId: "hashed" }`	47 / 25 / 27 %	**48** / 25 / 27 %	1 shard
`{ fromWalletId: 1, createdAt: 1 }`	1 chunk, corrigé à la main	33 / 34 / 33 %	1 shard
`{ walletCle: "hashed" }`	**32 / 33 / 35 %**, sans correction	**32 / 34 / 34 %**	1 shard (par `walletCle`)

Le hotspot de la date est démontré par les chiffres : une clé monotone envoie toutes les écritures nouvelles dans le dernier chunk. Au pic de 1 200 tx/s, un seul shard recevrait tout : le sharding ne servirait à rien pour ce qui le justifie.

Pourquoi une clé à plages met tout sur un shard. Une collection shardée vide commence avec un seul chunk. Nos 60 000 transactions pèsent environ 15 Mo, sous les 128 Mo d'un chunk : rien n'est découpé. Nous avons corrigé par deux `sh.splitAt` et deux `sh.moveChunk`. Juste après, nos comptages affichaient 100 000 documents au lieu de 60 000 : le shard de départ garde une copie des documents déplacés (des **orphelins**), supprimée après 900 s. `$shardedDataDistribution` les sépare des documents possédés : 40 000 orphelins, puis 0 après le délai (`preuves/06-orphelins.txt`, `preuves/06-orphelins-apres.txt`).

Pourquoi la clé hachée sur l'émetteur est déséquilibrée. Les 13 505 dépôts (`CASH_IN`, 22,5 %) n'ont pas d'émetteur : ils ont tous la même valeur de clé, donc tombent dans le même chunk, qu'on ne peut pas découper. À la cible, cela ferait environ 90 millions de dépôts par an au même endroit. D'où notre clé `walletCle` : l'émetteur, ou le destinataire quand il n'y a pas d'émetteur. **0 document sans valeur de clé**, une répartition de 32 à 35 % sans aucune correction.

Requêtes ciblées ou dispersées (lu dans `explain()` à travers le mongos) : une requête n'est ciblée que si elle contient la clé. Avec `walletCle`, l'historique d'un wallet touche un seul shard (409 documents examinés pour 409 rendus) ; une transaction par `_id`, les paiements d'un marchand ou les transactions d'un jour interrogent les trois shards. Le CA par marchand est découpé : chaque shard fait un `$group` partiel (150 lignes au plus), et le mongos fusionne. **Le marchand qui fait 25 % du volume** (M00044) se répartit sur les trois shards (31,8 / 32,5 / 35,7 %), puisque ses paiements partent de wallets différents ; une clé sur `merchantId` aurait au contraire mis un quart des écritures sur une seule valeur.

Recommandation. Pas de sharding à 18 mois (Partie 0) ; nous le déclencherons au premier de ces seuils : 490 à 600 M transactions par an sur des nœuds de 64 Go, 1 à 2 To par Replica Set, ou des écritures au-delà de 50 % de la capacité du primary. **Ce jour-là, la clé sera `{ walletCle: "hashed" }`**, et nous ajoutons `walletCle` à chaque transaction **dès maintenant**, pour ne pas réécrire 400 millions de documents plus tard. Conséquence à préparer : sur une collection shardée, l'index unique d'idempotence doit commencer par la clé et deviendrait `{ walletCle: 1, idempotencyKey: 1 }`.

9 Partie 7 · Architecture proportionnée (ADR-003)

9.1 Notre cible : PME, un Replica Set et un read model

Ce sont nos chiffres de la Partie 0 qui fixent la cible, pas une préférence :

— **Pas « Startup »** : un serveur seul ne survit pas à sa propre panne (99,95 % et RPO 0 exigés), et les transactions multi-documents demandent un Replica Set. Nous avons mesuré 0 écriture majority perdue sur panne du primary avec trois nœuds.

— **Pas « Enterprise »** : le working set (25,9 Go) tient dans le cache d'un nœud de 64 Go (40 % de la RAM) ; 307,9 Go par an avec index, loin de la zone de 1 à 2 To par Replica Set ; 400 M transactions par an pour un seuil de sharding à 490 M.

— **PME** : un Replica Set de trois nœuds pour l'argent, et un **read model** pour le tableau de bord et les règlements, pour que les agrégats ne pèsent pas sur le primary.

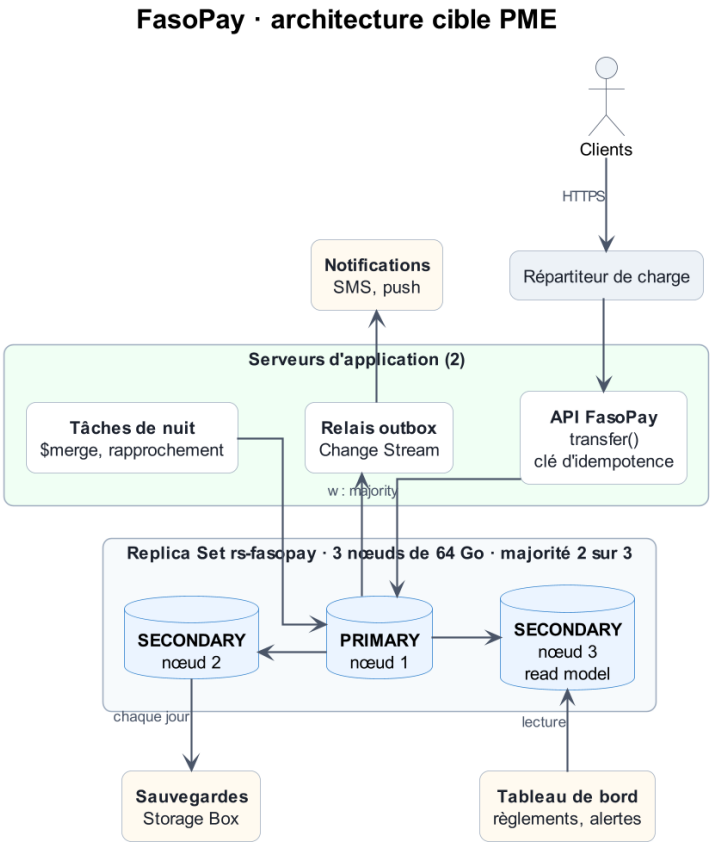


Figure 1 – Architecture cible PME de FasoPay (source PlantUML : `rapport/diagramme-architecture.puml`).

Le chemin d'un virement : l'API appelle `transfer()`, qui écrit les 6 documents en une transaction `w: "majority"` ; le relais envoie les notifications à partir de `outbox` (bonus B2) ; chaque nuit, un `$merge` recalcule le read model, lu sur un secondary, et le ledger est rapproché des soldes (E1).

9.2 Composants retenus, et le chiffre qui rend chacun nécessaire

Composant	Le chiffre qui le rend nécessaire
Replica Set de 3 nœuds	RPO 0 et 99,95 % : 0 écriture perdue sur panne du primary, contre 97 en `w: 1`
Nœuds de 64 Go de RAM	working set de 25,9 Go pour un cache de 31,5 Go (22 % de marge)
2 × 1 To NVMe en RAID 1	198,5 Go par an ; 2 ans gardés en ligne = 397 Go sur 700 Go utilisables (57 %), archives au-delà
`transfer()` avec clé d'idempotence	21 clés rejouées et 65 rejeux (E4) ; sans clé, 633 025 FCFA débités en trop
Client réglé à 1 s, `retryWrites`	interruption d'une panne ramenée de 21,2 s à 10,6 s
Read model `ca_marchands`	615 Mo de données nouvelles par jour : un `$merge` quotidien suffit, et le primary (330 virements/s mesurés pour 2 400 nécessaires) n'est pas sollicité
Rapprochement nocturne	6 soldes incohérents, 138 000 FCFA d'écart absolu (E1)
Alertes plafond → `fraud_alerts`	5 wallets qui découpent leurs transferts (E2)
Relais outbox par Change Stream	un seul consommateur des événements ; 0 perdu, 0 doublon après `kill -9` (B2)
Sauvegardes hors des nœuds	une suppression par erreur est répliquée en millisecondes : il faut une copie et l'oplog pour revenir avant l'erreur
2 serveurs d'application	un seul rendrait inutile la disponibilité du Replica Set

9.3 Composants écartés, avec le seuil qui les ferait revenir

Composant	Pourquoi pas aujourd'hui	Nous le réintroduisons quand…
Sharding	400 M tx/an, working set à 40 % de la RAM	490 M tx/an (working set à 50 % de 64 Go), ou écritures au-delà de 50 % du primary au test de charge
Kafka	un seul consommateur, servi par un Change Stream	un 2e consommateur indépendant doit relire les événements au-delà de l'oplog, ou le relais prend plus de 60 s de retard au pic
Spark / Data Lake	615 Mo par jour ; E3 en 23 ms	plus de 1 To à transformer par cycle, ou des sources hors MongoDB à croiser
Redis (solde)	3 M soldes = 0,43 Go en cache, lecture en 0 ms médiane ; un cache créerait un second solde qui peut diverger	lecture de solde au-delà de 20 ms au p99 sous charge, ou plus de 20 000 lectures/s
Lambda	pas de double besoin temps réel + recalcul lourd	les deux besoins à la fois
Nœuds de 128 Go	27 octets par entrée d'index	index au-delà de 45 octets par entrée, ou working set au-delà de 60 % de la RAM

9.4 Sécurité et conformité

Authentification et rôles minimaux, prouvés. Sur un serveur démarré avec authentification obligatoire, nous avons créé trois rôles, un par usage, et testé chaque utilisateur sur ce qu'il doit et ne doit pas pouvoir faire : **18 vérifications sur 18** (`preuves/07-securite.txt`).

Rôle	Peut	Ne peut pas (refusé, code 13)
`fasopay_application`	insérer transactions, lignes de ledger et événements ; mettre à jour un solde	**modifier ou supprimer une ligne de ledger**, supprimer un wallet, lire le read model
`fasopay_relais`	lire et marquer les événements `outbox`, écrire une notification	modifier un solde, lire le ledger
`fasopay_tableau`	lire le read model	lire les transactions, écrire

Le ledger devient ainsi un journal en **ajout seul** : même une API compromise ne peut pas réécrire l'historique de l'argent.

Chiffrement. En transit : TLS partout (clients, API, et entre les nœuds avec `net.tls.mode: requireTLS`). Au repos : le moteur de stockage chiffré n'existe que dans l'édition Enterprise de MongoDB ; avec l'édition Community, nous chiffrons les disques au niveau du système et les sauvegardes avant leur envoi. **Journal d'audit :** la documentation de MongoDB 8.0 ne le décrit que pour Enterprise et Atlas ; nous nous appuyons sur le ledger en ajout seul, le journal du serveur conservé hors des nœuds et un utilisateur par rôle. **Conservation :** les lignes de ledger ne sont jamais supprimées ; au-delà de 2 ans, elles passent dans une archive chiffrée. Nous ne fixons pas nous-mêmes la durée légale.

Autorités à consulter : la **Commission de l'Informatique et des Libertés (CIL)** du Burkina Faso, autorité de protection des données personnelles, notamment parce que nos serveurs seraient hors du pays ; et la **BCEAO**, qui délivre l'agrément des émetteurs de monnaie électronique dans l'UEMOA.

9.5 Coût indicatif mensuel

Prix publics hors taxes relevés en novembre 2026 chez Hetzner, au taux fixe de 1 € = 655,957 FCFA.

Poste	€ HT / mois	soit FCFA
3 serveurs dédiés EX63 (64 Go ECC, 2 × 1 To NVMe) : le Replica Set	447,00	293 213
Storage Box 5 To : sauvegardes et archives	10,90	7 150
2 serveurs cloud CX33 avec IPv4 : API, relais, tâches, surveillance	17,98	11 794
Répartiteur de charge LB11 avec IPv4	7,49	4 913
Total	**483,37**	**317 070**

Plus 222 € d'installation une seule fois, soit environ 145 622 FCFA. **Hypothèses :** prix sans remise et susceptibles de changer ; serveur EX44 écarté (59 €) car sans mémoire ECC ; nœuds dans des centres de données différents ; hors personnel, SMS, licence Enterprise, et hors hébergement au Burkina Faso si la CIL ou la BCEAO l'exigent (nous n'avons pas de prix vérifié à donner) ; latence Ouagadougou–Europe non mesurée.

ADR-003 (décision). Cible PME : un Replica Set de 3 nœuds de 64 Go, un read model et un relais outbox par Change Stream, **sans Kafka, Spark, Redis ni sharding à 18 mois**. Rejetés : l'architecture Enterprise (au moins 15 machines au lieu de 3, sans qu'aucun seuil ne soit franchi), un serveur seul (incompatible avec 99,95 %), Redis devant les soldes et Kafka dès le démarrage. À revoir aux seuils du tableau des écartés, et d'abord au test de charge. ADR complet dans `adr/ADR-003.md`.

10 Synthèse CTO : 5 priorités pour les 12 prochains mois

Notre verdict. FasoPay n'a pas besoin d'une plateforme de données, mais d'un Replica Set de 3 nœuds bien réglé, d'un `transfer()` utilisé partout et d'un contrôle quotidien de l'argent, pour environ **483 € HT par mois, soit environ 317 070 FCFA**.

1. **Réparer la confiance dans les soldes (mois 1).** Corriger les 6 soldes incohérents (138 000 FCFA d'écart absolu) par des écritures d'ajustement, en partant du ledger qui fait foi, puis lancer chaque nuit le rapprochement. *Réussite :* 0 écart chaque matin, et tout nouvel écart vu par la finance avant le support.
2. **Faire passer 100 % des flux d'argent par `transfer()` (mois 1 à 3)**, avec une clé d'idempotence fixée par le client. C'est la réponse aux rejeux de l'incident du 10 avril : un seul débit pour 3 rejeux, 30 virements sur 50 en concurrence, jamais de solde négatif. *Réussite :* 0 doublon, 0 solde négatif.
3. **Mettre en production le Replica Set sûr (mois 2 à 4) :** 3 nœuds de 64 Go sur des sites différents, `w: "majority"` pour l'argent, client réglé à 1 s, sauvegardes avec restauration **testée**, TLS, disques chiffrés et les trois rôles minimaux.
4. **Outiller la conformité et la finance (mois 3 à 6) :** alerte automatique sur le découpage sous le plafond (5 wallets aujourd'hui), read model pour le tableau de bord et les règlements, relais outbox pour les notifications ; consulter dès maintenant la CIL et intégrer les exigences de la BCEAO.
5. **Mesurer avant de grandir (mois 4 à 12) :** test de charge sur le matériel cible (il faut 2 400 virements/s, nous en mesurons 330 sur un portable), puis suivi mensuel des seuils : working set au-delà de 60 % de la RAM, index au-delà de 45 octets par entrée, volume vers 490 M tx/an. La clé de sharding est déjà choisie.

Ce que nous ne ferons PAS

Nous ne ferons pas	Parce que (nos chiffres)	Sauf si
Kafka	un seul consommateur des événements	un 2e consommateur doit les relire
Spark ou un Data Lake	615 Mo nouveaux par jour, E3 en 23 ms	plus de 1 To par cycle
Redis devant les soldes	0,43 Go en cache, lecture en 0 ms	p99 au-delà de 20 ms sous charge
Sharding	400 M tx/an, working set à 40 % de la RAM	490 M tx/an ou test de charge insuffisant
Modifier ou supprimer le ledger	le ledger est la preuve de l'argent	jamais
Changer de base de données	MongoDB couvre transactions, réplication, Change Streams et agrégats, et tout notre POC tourne dessus	jamais sans mesure qui le justifie

La décision à prendre maintenant : valider la cible PME (ADR-003) et commander le test de charge sur le matériel cible ; c'est le seul chiffre qui pourrait changer cette architecture.

11 Bonus B2 · Relais Outbox par Change Stream

`transfer()` écrit l'événement `outbox` dans la même transaction que le virement : un virement validé a toujours son événement. Il reste à l'**envoyer** sans en perdre un et sans en envoyer deux, même si le relais meurt n'importe quand. Notre relais (`code/B2-relais.js`) ouvre un Change Stream sur `outbox` avec un filtre appliqué **par le serveur** (seules les insertions `PENDING` lui parviennent). Pour chaque événement, dans cet ordre : (1) il écrit la notification avec **le même `_id` que l'événement** ; (2) il passe l'événement à `SENT` ; (3) il sauvegarde le **jeton de reprise**. Au redémarrage, il reprend avec `resumeAfter`.

Scénario	Ce qui s'est passé	Résultat
`kill -9` au milieu de 1 000 virements	relais tué après 324 notifications ; 410 événements s'accumulent pendant 3 s ; reprise « après 323 »	1 relivraison reconnue ; 1 000 événements, 1 000 notifications, **0 perdu, 0 doublon**
Arrêt juste après la 5e notification, avant `SENT` et avant le jeton	1 005 notifications pour 1 004 événements `SENT` au moment de l'arrêt	1 relivraison reconnue ; 1 020 / 1 020, **0 perdu, 0 doublon**

Chaque scénario se termine par 6 vérifications, toutes OK (`preuves/B2-relais-outbox.txt`). Le `kill -9` est tombé, par hasard, dans la fenêtre dangereuse ; le second scénario la provoque exprès, pour que la preuve ne dépende pas du hasard.

Pourquoi « au moins une fois ». Le jeton est sauvegardé *après* la notification : si le relais meurt entre les deux, le Change Stream relivre l'événement au redémarrage. Rien n'est perdu, mais un événement peut arriver deux fois. Sauvegarder le jeton avant ferait l'inverse, et perdrait un événement à chaque arrêt dans cette fenêtre : pour de l'argent, nous préférons recevoir deux fois que jamais.

Ce qui rend le résultat « exactement une fois ». La notification porte le même `_id` que l'événement, et `_id` est unique : à la relivraison, `upsert` avec `$setOnInsert` trouve la notification déjà écrite et ne change rien. La livraison est « au moins une fois », l'effet « exactement une fois » : c'est le principe de la clé d'idempotence de `transfer()`, appliqué au relais. **Limites :** au-delà de la fenêtre de l'oplog, la reprise doit repartir des événements encore `PENDING` ; un vrai fournisseur de SMS devrait recevoir lui aussi l'`_id` comme clé.

12 Conclusion et limites du POC

Les quatre demandes du CTO ont une réponse prouvée. **Le modèle** rejette ce qui est faux et tient la comptabilité en partie double. **L'enquête** trouve les 6 soldes divergents et chiffre leur écart, identifie 5 wallets qui découpent leurs transferts, et montre que l'idempotence a tenu pendant l'incident du 10 avril. **Les virements** ne peuvent plus ni doubler ni se perdre : nous l'avons prouvé par l'abort, la concurrence, trois pannes réelles et un relais tué en pleine rafale. **L'architecture** est proportionnée à nos chiffres, chaque composant écarté a son seuil de retour, et elle coûte environ 317 070 FCFA par mois.

Ce que notre POC ne prouve pas, et que nous ne prétendons pas : le débit au pic (330 virements/s sur un portable pour 2 400 nécessaires, à trancher par un test de charge) ; la taille des index à 400 millions de documents (d'où notre seuil de 45 octets) ; les performances d'un vrai réseau, puisque nos nœuds et nos shards partagent une machine ; la cause des 6 soldes divergents, dont le « lost update » n'est qu'une explication possible ; et les exigences de la CIL et de la BCEAO.

Annexes

Annexe A Feuille de dimensionnement
Annexe B Journal de montage
Annexe C Code
Annexe D Preuves
Annexe E Déclaration d'usage de l'IA

Annexe A · Feuille de dimensionnement

Groupe : 07 · empreinte (`datasetSha256`, début) : 9b0b9a40

Unités : 1 Mo = 1 000 000 octets, 1 Go = 1 000 000 000 octets. Mesures : `preuves/00-dimensionnement.txt` (commandes dans `code/00-dimensionnement.js`).

Cible à 18 mois (donnée du sujet)

3 000 000 wallets · 400 000 000 transactions/an · pic 1 200 tx/s · disponibilité 99,95 % · RPO 0

1. Mesures sur notre jeu (POC)

Collection	Documents	Taille BSON moyenne (octets)	Données (Mo)	Index (Mo)
transactions	60 000	253,02	15,18	5,05
ledger_entries	141 939	130,19	18,48	7,48
wallets	2 000	144	0,29	0,11

Les tailles moyennes viennent de `$bsonSize` sur toute la collection ; un échantillon aléatoire de 1 000 documents donne 252,44 (transactions) et 130,30 (ledger). « Données » est la taille BSON non compressée (`size`) ; sur disque, la compression ramène les transactions à 7,98 Mo et le ledger à 9,27 Mo (`storageSize`). Les index sont ceux de la Partie 3 (3 sur `transactions`, 2 sur `ledger_entries`).

Lignes de ledger par transaction (mesuré) : 2,36565 (141 939 / 60 000), dont 1,19808 jambes de wallet (71 885 / 60 000).

Documents écrits par transaction métier (mesuré sur les 56 484 transactions qui ont un ledger) : 1 transaction + 2,513 lignes de ledger + 1,273 soldes de wallet + 1 outbox = 5,79 en moyenne. Par type : P2P 6,03, CASH_IN 5, CASH_OUT 6, BILL 6,02, MERCHANT_PAY 6,03.

2. Extrapolation

Grandeur	Calcul (formule + nombres)	Résultat
Volume brut/an (données)	400 M × (253,02 + 2,36565 × 130,19) = 400 M × 561,02 octets	224,4 Go
Facteur d'index retenu (et pourquoi)	(33 661 042 + 12 529 664) / 33 661 042 : données puis index de `transactions` et `ledger_entries`, mesurés après les index de la Partie 3. Ces deux collections portent 98 % des données.	×1,37
Volume/an avec index	224,4 Go × 1,3723	307,9 Go
Écritures/s au pic	1 200 × 6 documents (1 transaction + 2 lignes de ledger + 2 soldes + 1 outbox, ADR-001) ; en plus 8 entrées d'index par virement (3 sur `transactions`, 4 sur `ledger_entries`, 1 sur `outbox`)	7 200 documents/s et 9 600 entrées d'index/s

Grandeur	Calcul (formule + nombres)	Résultat
Working set (30 jours chauds + index chauds)	400 M × 30 / 365 = 32,88 M transactions × 561,02 o = 18,44 Go ; index chauds 18,44 × 0,3723 = 6,87 Go ; 3 M wallets × 144 o = 0,43 Go ; index des wallets 3 M × 55,3 o = 0,17 Go	25,9 Go
RAM par nœud (cache WiredTiger ≈ 50 % de (RAM − 1 Go))	50 % × (RAM − 1) ≥ 25,9 donc RAM ≥ 2 × 25,9 + 1 = 52,8 Go : nœud de 64 Go (cache 31,5 Go, marge 22 %)	64 Go
Disque par nœud et horizon avant saturation	données sur disque : 224,4 Go × 0,5124 (compression mesurée : 17 248 256 / 33 661 042) = 115,0 Go ; index : 224,4 × 0,3723 = 83,5 Go ; total 198,5 Go/an ; disque utilisable : 2 To × 70 % = 1 400 Go ; 1 400 / 198,5	7,1 ans
Facteur de pic	moyenne : 400 M ÷ (365 × 86 400 s) = 12,7 tx/s ; pic ÷ moyenne = 1 200 ÷ 12,7	×95 : nous dimensionnons sur le pic, pas sur la moyenne
Rétention sur 5 ans	sur disque : 5 × 198,5 Go = 0,99 To ; non compressé, avec index : 5 × 307,9 Go = 1,54 To	tient sur 1,4 To utilisables pendant 7 ans, mais entre dans la zone de 1 à 2 To par Replica Set à partir de laquelle on envisage de répartir : nous archivons au-delà de 2 ans

2 bis. Un chiffre qui contredit l'architecture : la taille des index

Mesuré : 12 529 664 octets d'index pour 463 878 entrées (3 index × 60 000 transactions + 2 index × 141 939 lignes), soit **27 octets par entrée**. C'est peu : à 60 000 documents, un index reste plus compact qu'à 400 millions. Une règle de prudence courante compte 40 à 60 octets par entrée (chapitre 11). Nous refaisons donc le calcul avec ces valeurs :

Hypothèse	Index par transaction métier (7,73 entrées)	Index chauds (30 jours)	Working set	RAM nécessaire
Mesuré (27 o)	209 o	6,9 Go	25,9 Go	52,8 Go : 64 Go suffit
Prudence basse (40 o)	309 o	10,2 Go	29,2 Go	59,4 Go : 64 Go juste
Prudence haute (60 o)	464 o	15,3 Go	34,3 Go	69,6 Go : **64 Go ne suffit plus**

Le nœud de 64 Go tient tant que les index restent sous **49 octets par entrée** : au-delà, le working set dépasse le cache de 31,5 Go. Nous gardons donc 64 Go comme base, avec un seuil de surveillance : si les index dépassent 45 octets par entrée en production, ou si le working set dépasse 60 % de la RAM, nous passons à des nœuds de 128 Go. C'est une montée en gamme, pas un sharding.

2 ter. Un deuxième chiffre qui contredit : le débit mesuré

Nous avons mesuré `transfer()` sur notre machine (Partie 4, `preuves/04-debit.txt`) : **125 virements/s** avec un client, **330 virements/s** avec 4 clients en parallèle (1 978 écritures de documents/s), p95 à 17 ms. La cible est de 1 200 virements/s au pic, et la règle que nous retenons demande que les écritures restent sous **50 % de la capacité du primary** : il faudrait donc un primary capable d'environ **2 400 virements/s**, soit plus de 7 fois ce que nous mesurons.

Ce chiffre ne condamne pas l'architecture, mais il ne la prouve pas non plus : la mesure a été faite sur un ordinateur portable, avec les clients sur la même machine que le serveur et un seul nœud. **Décision :** avant la mise en production, un test de charge sur le matériel cible est obligatoire. Si le primary n'y atteint pas 2 400 virements/s, nous montons d'abord en gamme (CPU, disques plus rapides) ; si cela ne suffit pas, nous répartissons les écritures (sharding, Partie 6).

3. Verdict (une phrase chacun, avec un SEUIL chiffré)

Nous décidons chaque composant sur trois critères : le volume par Replica Set, la part de la RAM prise par le working set, et la part du débit du primary prise par les écritures. Pour Kafka, ce n'est pas le débit qui compte, mais le nombre de consommateurs indépendants et le besoin de rejeu.

Question	Réponse	Seuil de bascule (chiffré, avec votre ordre de grandeur)
Un seul replica set suffit-il ?	Oui à 18 mois : 0,31 To par an non compressé (sous la zone de 1 à 2 To) ; working set de 25,9 Go sur 64 Go, soit 40 % de la RAM (54 % dans l'hypothèse prudente de la section 2 bis) ; pour les 7 200 écritures par seconde au pic, nous avons mesuré 330 virements/s sur un portable (section 2 ter) : le critère n'est pas encore vérifié sur le matériel cible.	Working set au-delà de 60 % de la RAM ; ou plus de 1 à 2 To par replica set ; ou écritures au-delà de 50 % du débit du primary, mesuré par un test de charge sur le matériel cible (il faut au moins 2 400 virements/s).
Sharding ?	Non : aucun des trois critères n'est franchi à 18 mois. Nous ne le déclencherons que sur cette feuille et un test de charge.	Sur des nœuds de 64 Go : entre 490 M tx/an (working set à 50 % de la RAM) et 600 M tx/an (60 %). En volume : 1 To non compressé atteint en 3,2 ans, 2 To en 6,5 ans.
Kafka ?	Non au démarrage : un seul consommateur des événements `outbox` (le relais de notifications), qu'un Change Stream avec jeton de reprise suffit à servir.	Dès qu'un **deuxième consommateur indépendant** doit relire les événements au-delà de la fenêtre de l'oplog (par exemple la détection de fraude ou la comptabilité), ou qu'il faut un tampon devant le pic. Probable à la cible : nous le tranchons en Partie 7.
Spark / Data Lake ?	Non : 224,4 Go de données par an, soit environ 615 Mo par jour ; un `$merge` quotidien vers un read model suffit. E3 prend 23 ms sur le POC (`03-explain.txt`).	Plusieurs To à transformer par cycle, ou des sources hétérogènes hors MongoDB (opérateurs télécom, banques partenaires).
Redis pour le solde ?	Non : les 3 M soldes pèsent 0,43 Go et tiennent dans le cache ; une lecture exacte par index prend 0 ms en médiane (Q3, `03-explain.txt`). Nous n'ajoutons pas de cache sans mesure qui le justifie.	Une lecture de solde au-delà de 20 ms au p99, mesurée sous charge, ou plus de 20 000 lectures de solde par seconde soutenues.
Lambda ?	Non : nous n'avons pas de besoin cumulé de temps réel **et** de recalcul historique lourd.	Les deux besoins à la fois, sans qu'un seul moteur puisse les servir.

4. Hypothèses (liste)

Mesurées sur le POC, puis extrapolées : 1. Tailles moyennes : 253,02 octets par transaction, 130,19 par ligne de ledger, 144 par wallet ; 2,36565 lignes de ledger par transaction. Nous supposons qu'elles restent les mêmes à 3 M wallets (identifiants de même forme). 2. Compression sur disque (0,5124) et facteur d'index (0,3723), mesurés sur le POC et supposés constants. Le facteur d'index est probablement optimiste : voir la section 2 bis. 3. Écritures par transaction métier : 6 dans le cas le plus lourd, 5,79 en moyenne mesurée.

Non testées : 4. Unités décimales : 1 Mo = 10^6 octets, 1 Go = 10^9 octets. 5. Fenêtre chaude de 30 jours, trafic réparti sur l'année (30 / 365 du volume annuel). Un pic de 2 h chaque 1er du mois représenterait 8,64 M transactions par mois, soit 26 % du volume : nous supposons qu'il reste dans le mois chaud. 6. Nœud de référence : 64 Go de RAM, disque de 2 To dont 70 % utilisables ; l'oplog n'est pas compté. 7. Les 3 M wallets sont tous dans le working set ; `customers` (0,46 Go) et `merchants` (0,013 Go) n'y sont pas. 8. Les documents `outbox` sont supprimés après envoi : ils n'entrent pas dans le volume annuel (un document pèse 109 octets, mesuré). 9. Volume constant de 400 M tx/an ; archivage des données de plus de 2 ans. 10. Taille des index par entrée : 27 octets mesurés, 40 à 60 en hypothèse prudente (section 2 bis). 11. Débit d'écriture d'un nœud : mesuré sur un ordinateur portable seulement (125 virements/s avec 1 client, 330 avec 4) ; la capacité d'un serveur de production n'est pas testée (section 2 ter).

Annexe B · Journal de montage

Les commandes qui construisent nos environnements, dans l'ordre, telles que nous les avons tapées. Le journal complet, avec l'objectif de chaque étape, le résultat attendu et la date de chaque preuve, est dans `docs/journal-montage.md` (dans le ZIP).

1. Environnement de départ

```
git --version
```

```
docker --version
```

```
docker info --format "{{.MemTotal}}"
```

2. Données du groupe et vérification

```
Get-ChildItem data\*.ndjson | ForEach-Object {
        "{0}  {1}  {2}" -f (Get-FileHash $_.FullName -Algorithm SHA256).Hash.ToLower().Substring
(0,8), $_.Name, $_.Length
}
```

3. Serveur MongoDB (D1)

```
docker run -d --name mongo-rs1 -p 27017:27017 -v mongo-rs1-data:/data/db mongo:8.0 --replSet rs0
```

```
docker exec mongo-rs1 mongod --version
```

```
docker exec mongo-rs1 mongosh --version
```

4. Initialisation du Replica Set (D1)

```
docker exec mongo-rs1 mongosh --eval "rs.initiate({_id: 'rs0', members: [{_id: 0, host: '
localhost:27017'}]})"
```

5. Vérification du Replica Set (D1)

```
docker exec mongo-rs1 mongosh --eval "rs.status()"
```

```
docker exec mongo-rs1 date -u
```

```
docker exec mongo-rs1 mongosh --eval "rs.status()"
```

6. Copie des données dans le conteneur (D2)

```
docker cp data mongo-rs1:/tmp/data
```

7. Import des collections (D2)

```
            docker exec mongo-rs1 mongoimport --db fasopay_tp --collection customers --file /tmp/data/
customers.ndjson
```

```
            docker exec mongo-rs1 mongoimport --db fasopay_tp --collection wallets --file /tmp/data/wallets.
ndjson
```

```
            docker exec mongo-rs1 mongoimport --db fasopay_tp --collection merchants --file /tmp/data/
merchants.ndjson
```

```
            docker exec mongo-rs1 mongoimport --db fasopay_tp --collection transactions --file /tmp/data/
transactions.ndjson
```

```
            docker exec mongo-rs1 mongoimport --db fasopay_tp --collection ledger_entries --file /tmp/data/
ledger_entries.ndjson
```

8. Comptage des collections (preuve D2)

```
            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.getCollectionNames().forEach(function
(c){ print(c + ' : ' + db[c].countDocuments()) })"
```

```
            docker exec mongo-rs1 date -u
```

```
            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.getCollectionNames().forEach(function
(c){ print(c + ' : ' + db[c].countDocuments()) })"
```

9. Vérification des données (D3)

```
            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.transactions.aggregate([{`$group:{_id
:null, total:{`$sum:``$amount`}}}]).toArray()"
```

```
            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.transactions.aggregate([{`$match:{
status:'COMPLETED'}},{`$group:{_id:null, total:{`$sum:``$amount`}}}]).toArray()"
```

```
            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.wallets.aggregate([{`$group:{_id:null
, total:{`$sum:``$balance`}}}]).toArray()"
```

```
            Get-ChildItem data\*.ndjson | ForEach-Object { "{0}  {1}" -f (Get-FileHash $_.FullName -Algorithm
 SHA256).Hash.ToLower(), $_.Name }
```

```
            docker exec mongo-rs1 date -u
```

10. Création des collections vides (D4)

```
            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.createCollection('outbox')"
            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.createCollection('fraud_alerts')"

            docker exec mongo-rs1 mongosh fasopay_tp --quiet --eval "db.getCollectionNames().sort()"
```

11. Vérification des types après l'import

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
db.transactions.aggregate([{$group: {_id: {amount: {$type: "$amount"}, createdAt: {$type: "
$createdAt"}}, n: {$sum: 1}}}])
db.transactions.aggregate([{$group: {_id: {fee: {$type: "$fee"}, completedAt: {$type: "
$completedAt"}}, n: {$sum: 1}}}])
db.wallets.aggregate([{$group: {_id: {balance: {$type: "$balance"}, dailyLimit: {$type: "
$dailyLimit"}, version: {$type: "$version"}, updatedAt: {$type: "$updatedAt"}}, n: {$sum: 1}}}])
db.ledger_entries.aggregate([{$group: {_id: {amount: {$type: "$amount"}, balanceAfter: {$type: "
$balanceAfter"}, walletId: {$type: "$walletId"}, createdAt: {$type: "$createdAt"}}, n: {$sum: 1}}}])
db.customers.aggregate([{$group: {_id: {kycLevel: {$type: "$kycLevel"}, createdAt: {$type: "
$createdAt"}}, n: {$sum: 1}}}])
db.merchants.aggregate([{$group: {_id: {mcc: {$type: "$mcc"}, feePct: {$type: "$feePct"},
location: {$type: "$location"}, createdAt: {$type: "$createdAt"}}, n: {$sum: 1}}}])
```

12. Validateurs de la Partie 1

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

13. Mesures pour la modélisation (Partie 1)

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

14. Enquête (Partie 2)

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
db.transactions.aggregate([{ $match: { type: "MERCHANT_PAY" } }, { $out: { db: "fasopay_audit",
coll: "paiements" } }])
db.getSiblingDB("fasopay_audit").dropDatabase()
```

15. Index et explain (Partie 3)

```
            docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
            db.transactions.createIndex({ fromWalletId: 1, status: 1, createdAt: 1, amount: 1 }, { name: "
idx_sorties_wallet_jour" })
            db.ledger_entries.createIndex({ walletId: 1, createdAt: -1 }, { name: "idx_ledger_wallet_date" })
            db.transactions.createIndex({ idempotencyKey: 1 }, { unique: true, name: "uniq_idempotency_key"
})
```

```
            docker exec -it mongo-rs1 mongosh fasopay_tp
```

```
            db.transactions.createIndex({ type: 1, status: 1, merchantId: 1, amount: 1 }, { name: "
idx_refuse_ca_marchand" })
            db.transactions.dropIndex("idx_refuse_ca_marchand")
```

16. Mesures de dimensionnement (Partie 0)

```
            docker exec -it mongo-rs1 mongosh fasopay_tp
```

17. Base de test et tests des transactions (Partie 4)

```
            docker exec mongo-rs1 mkdir -p /tmp/code /tmp/preuves
            docker cp code/. mongo-rs1:/tmp/code/
```

```
            docker exec mongo-rs1 sh /tmp/code/04-lancer.sh tests
            docker cp mongo-rs1:/tmp/preuves/04-transaction.txt preuves/04-transaction.txt
            docker exec mongo-rs1 sh /tmp/code/04-lancer.sh concurrence
            docker cp mongo-rs1:/tmp/preuves/04-concurrence.txt preuves/04-concurrence.txt
            docker exec mongo-rs1 sh /tmp/code/04-lancer.sh lostupdate
            docker cp mongo-rs1:/tmp/preuves/04-lost-update.txt preuves/04-lost-update.txt
```

```
            docker cp code/. mongo-rs1:/tmp/code/
            docker exec mongo-rs1 sh /tmp/code/04-lancer.sh debit
            docker cp mongo-rs1:/tmp/preuves/04-debit.txt preuves/04-debit.txt
```

18. Replica Set à 3 nœuds (Partie 5)

```
            docker network create reseau-fasopay
```

```
            docker run -d --name fasopay-noeud1 --network reseau-fasopay -v fasopay-noeud1-data:/data/db
mongo:8.0 --replSet rs-fasopay --bind_ip_all --wiredTigerCacheSizeGB 0.25
            docker run -d --name fasopay-noeud2 --network reseau-fasopay -v fasopay-noeud2-data:/data/db
mongo:8.0 --replSet rs-fasopay --bind_ip_all --wiredTigerCacheSizeGB 0.25
            docker run -d --name fasopay-noeud3 --network reseau-fasopay -v fasopay-noeud3-data:/data/db
mongo:8.0 --replSet rs-fasopay --bind_ip_all --wiredTigerCacheSizeGB 0.25
```

```
            docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Networks}}"
            docker stats --no-stream --format "table {{.Name}}\t{{.MemUsage}}\t{{.CPUPerc}}"
```

```
            docker exec fasopay-noeud1 mongosh --quiet --eval "rs.initiate({_id: 'rs-fasopay', members: [{_id
: 0, host: 'fasopay-noeud1:27017'}, {_id: 1, host: 'fasopay-noeud2:27017'}, {_id: 2, host: 'fasopay-noeud3
:27017'}]})"
```

```
            docker cp code/. fasopay-noeud1:/tmp/code/
            docker exec fasopay-noeud1 sh /tmp/code/05-lancer.sh status
            docker cp fasopay-noeud1:/tmp/preuves/05-rs-status.txt preuves/05-rs-status.txt
```

```
            docker run -d --name fasopay-client --network reseau-fasopay mongo:8.0 sleep infinity
            docker cp code/. fasopay-client:/tmp/code/
            docker exec fasopay-client sh /tmp/code/05-lancer.sh primaire
            docker exec -d fasopay-client sh /tmp/code/05-lancer.sh boucle majority
            docker exec fasopay-client sh /tmp/code/05-lancer.sh panne majority; docker kill fasopay-noeud1
            docker exec fasopay-client sh /tmp/code/05-lancer.sh attendre majority
            docker start fasopay-noeud1
            docker exec fasopay-client sh /tmp/code/05-lancer.sh analyse majority
            docker cp fasopay-client:/tmp/preuves/05-failover-majority.txt preuves/05-failover.txt
```

```
            docker cp code/. fasopay-client:/tmp/code/
            docker exec fasopay-client sh /tmp/code/05-lancer.sh primaire
            docker exec -d fasopay-client sh /tmp/code/05-lancer.sh boucle majority-hb1
            docker exec fasopay-client sh /tmp/code/05-lancer.sh panne majority-hb1; docker kill fasopay-
noeud3
            docker exec fasopay-client sh /tmp/code/05-lancer.sh attendre majority-hb1
            docker start fasopay-noeud3
            docker exec fasopay-client sh /tmp/code/05-lancer.sh analyse majority-hb1
            docker cp fasopay-client:/tmp/preuves/05-failover-majority-hb1.txt preuves/05-failover-hb1.txt
```

```
            docker cp code/. fasopay-noeud2:/tmp/code/
            docker cp code/. fasopay-client:/tmp/code/
            docker exec fasopay-client sh /tmp/code/05-lancer.sh primaire
            docker exec -d fasopay-noeud2 sh /tmp/code/05-lancer.sh boucle-locale w1
            docker exec fasopay-noeud2 sh /tmp/code/05-lancer.sh panne w1; docker network disconnect reseau-
fasopay fasopay-noeud2
            docker exec fasopay-noeud2 sh /tmp/code/05-lancer.sh attendre w1
            docker network connect reseau-fasopay fasopay-noeud2
            docker exec fasopay-noeud2 sh /tmp/code/05-lancer.sh analyse w1
            docker exec fasopay-noeud2 sh /tmp/code/05-lancer.sh rollback w1
            docker cp fasopay-noeud2:/tmp/preuves/05-failover-w1.txt preuves/05-failover-w1.txt
            docker cp fasopay-noeud2:/tmp/preuves/05-rollback.txt preuves/05-rollback.txt
```

```
            docker cp code/. fasopay-client:/tmp/code/
            docker exec fasopay-client sh /tmp/code/04-lancer.sh tests rs3
            docker exec fasopay-client sh /tmp/code/04-lancer.sh concurrence rs3
            docker cp fasopay-client:/tmp/preuves/04-transaction-rs3.txt preuves/04-transaction-rs3.txt
            docker cp fasopay-client:/tmp/preuves/04-concurrence-rs3.txt preuves/04-concurrence-rs3.txt
```

19. Cluster shardé (Partie 6)

```
            docker network create reseau-cluster
            docker run -d --name fasopay-cfg --network reseau-cluster -v fasopay-cfg-data:/data/configdb
mongo:8.0 --configsvr --replSet rs-cfg --port 27017 --bind_ip_all --wiredTigerCacheSizeGB 0.25
            docker run -d --name fasopay-shard1 --network reseau-cluster -v fasopay-shard1-data:/data/db
mongo:8.0 --shardsvr --replSet rs-shard1 --port 27017 --bind_ip_all --wiredTigerCacheSizeGB 0.25
            docker run -d --name fasopay-shard2 --network reseau-cluster -v fasopay-shard2-data:/data/db
mongo:8.0 --shardsvr --replSet rs-shard2 --port 27017 --bind_ip_all --wiredTigerCacheSizeGB 0.25
            docker run -d --name fasopay-shard3 --network reseau-cluster -v fasopay-shard3-data:/data/db
mongo:8.0 --shardsvr --replSet rs-shard3 --port 27017 --bind_ip_all --wiredTigerCacheSizeGB 0.25
```

```
            docker ps --format "table {{.Names}}    {{.Status}}     {{.Networks}}"
            docker stats --no-stream --format "table {{.Name}}      {{.MemUsage}}"
```

```
            docker exec fasopay-cfg mongosh --quiet --eval "rs.initiate({_id: 'rs-cfg', configsvr: true,
members: [{_id: 0, host: 'fasopay-cfg:27017'}]})"
            docker exec fasopay-shard1 mongosh --quiet --eval "rs.initiate({_id: 'rs-shard1', members: [{_id:
 0, host: 'fasopay-shard1:27017'}]})"
            docker exec fasopay-shard2 mongosh --quiet --eval "rs.initiate({_id: 'rs-shard2', members: [{_id:
 0, host: 'fasopay-shard2:27017'}]})"
            docker exec fasopay-shard3 mongosh --quiet --eval "rs.initiate({_id: 'rs-shard3', members: [{_id:
 0, host: 'fasopay-shard3:27017'}]})"
```

```
            docker run -d --name fasopay-mongos --network reseau-cluster mongo:8.0 mongos --configdb rs-cfg/
fasopay-cfg:27017 --bind_ip_all --port 27017
            docker exec fasopay-mongos mongosh --quiet --eval "printjson(sh.addShard('rs-shard1/fasopay-
shard1:27017')); printjson(sh.addShard('rs-shard2/fasopay-shard2:27017')); printjson(sh.addShard('rs-shard3/
fasopay-shard3:27017'))"
```

```
            docker cp code/. fasopay-mongos:/tmp/code/
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh cluster
            docker cp fasopay-mongos:/tmp/preuves/06-cluster.txt preuves/06-cluster.txt
```

```
            docker exec fasopay-mongos mkdir -p /tmp/data
            docker cp data/transactions.ndjson fasopay-mongos:/tmp/data/transactions.ndjson
            docker cp code/. fasopay-mongos:/tmp/code/
```

```
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh cle date
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh cle wallet
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh cle hash
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh cle composee
            docker cp fasopay-mongos:/tmp/preuves/06-cle-date.txt preuves/06-cle-date.txt
            docker cp fasopay-mongos:/tmp/preuves/06-cle-wallet.txt preuves/06-cle-wallet.txt
            docker cp fasopay-mongos:/tmp/preuves/06-cle-hash.txt preuves/06-cle-hash.txt
            docker cp fasopay-mongos:/tmp/preuves/06-cle-composee.txt preuves/06-cle-composee.txt
```

```
            docker cp code/. fasopay-mongos:/tmp/code/
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh orphelins
            docker cp fasopay-mongos:/tmp/preuves/06-orphelins.txt preuves/06-orphelins.txt
```

```
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh orphelins
            docker cp fasopay-mongos:/tmp/preuves/06-orphelins.txt preuves/06-orphelins-apres.txt
```

```
            docker cp code/. fasopay-mongos:/tmp/code/
            docker exec fasopay-mongos sh /tmp/code/06-lancer.sh cle cible
            docker cp fasopay-mongos:/tmp/preuves/06-cle-cible.txt preuves/06-cle-cible.txt
```

20. Sécurité : rôles minimaux (Partie 7)

```
            docker run -d --name fasopay-securite -e MONGO_INITDB_ROOT_USERNAME=admin-test -e
MONGO_INITDB_ROOT_PASSWORD=test-admin-2026 mongo:8.0 --wiredTigerCacheSizeGB 0.25
            docker cp code/. fasopay-securite:/tmp/code/
            docker exec fasopay-securite sh /tmp/code/07-lancer.sh securite
            docker cp fasopay-securite:/tmp/preuves/07-securite.txt preuves/07-securite.txt
```

21. Diagramme d'architecture (Partie 7)

```
            docker run --rm -v "${PWD}/rapport:/data" plantuml/plantuml -charset UTF-8 -tpng /data/diagramme-
architecture.puml
```

22. Bonus B2 : relais Outbox par Change Stream

```
docker start mongo-rs1
docker cp code/. mongo-rs1:/tmp/code/
docker exec mongo-rs1 sh /tmp/code/B2-lancer.sh relais
docker cp mongo-rs1:/tmp/preuves/B2-relais-outbox.txt preuves/B2-relais-outbox.txt
```

Annexe C · Code

Tous nos scripts sont dans le dossier `code/` du ZIP : les fichiers `.js` se lancent dans `mongosh`, les fichiers `.sh` dans un conteneur, et chacun écrit sa preuve (commande, date, sortie brute).

Fichier	Rôle	Partie
`00-dimensionnement.js`	mesures de dimensionnement, en lecture seule	0
`01-validateurs.js`, `01-mesures-modelisation.js`	validateurs, tests de rejet, mesures de modélisation	1
`02-enquete.js`	pipelines E1 à E4, commentés	2
`03-index.js`	index et `explain()` avant / après	3
`transfer.js`	**la fonction `transfer()`**, au contrat de l'énoncé	4
`04-*.js`, `04-lancer.sh`	base de test, tests, concurrence, lost update, débit	4, 5
`05-boucle.js`, `05-lancer.sh`	pannes du primary, rollback	5
`06-cles.js`, `06-lancer.sh`	clés de sharding, orphelins	6
`07-securite.js`, `07-lancer.sh`	rôles minimaux	7
`B2-relais.js`, `B2-outils.js`, `B2-lancer.sh`	relais outbox par Change Stream	B2

Annexe D · Preuves

Les sorties brutes sont dans le dossier `preuves/` du ZIP ; `preuves/LISEZMOI.md` dit, pour chaque fichier, ce qu'il démontre et quelle partie le cite.

Partie	Fichiers de `preuves/`
Contexte, D1 à D4	`00-rs-status-1-noeud`, `00-import-comptages`, `00-collections`, `00-fingerprint`, `00-types`
Partie 0	`00-dimensionnement`
Partie 1	`01-validator-rejet`, `01-mesures-modelisation`
Partie 2	`02-enquete`
Partie 3	`03-explain`
Partie 4	`04-transaction`, `04-concurrence`, `04-lost-update`, `04-debit` (et `-rs3`)
Partie 5	`05-rs-status`, `05-failover`, `05-failover-hb1`, `05-failover-w1`, `05-rollback`
Partie 6	`06-cluster`, `06-cle-date`, `06-cle-wallet`, `06-cle-hash`, `06-cle-composee`, `06-cle-cible`, `06-orphelins`, `06-orphelins-apres`
Partie 7	`07-securite`
Bonus B2	`B2-relais-outbox`

Annexe E · Déclaration d'usage de l'IA

Nous avons utilisé l'assistant Claude (Anthropic) tout au long du TP : pour préparer les scripts mongosh et les lanceurs, relire nos chiffres, proposer des brouillons de rédaction et la mise en forme LaTeX. Toutes les commandes ont été exécutées par nous, sur notre machine, et chaque chiffre de ce rapport vient de nos preuves, que nous avons relues. L'outil s'est trompé plusieurs fois ; voici les erreurs que nous avons repérées et corrigées :

1. **Partie 2, E3.** La requête de courbe de Pareto proposée a échoué sur notre serveur (`$documentNumber must be specified with a top level sortBy expression with exactly one element`), car elle triait sur deux champs. Nous l'avons corrigée (tri sur `ca` seul, après avoir vérifié que les 150 CA sont distincts) et refait la session.
2. **Partie 0.** L'outil nous a fait utiliser `Object.bsonsize()`, qui n'existe pas dans mongosh 2.14.0 (`TypeError` visible dans `preuves/00-dimensionnement.txt`). Nous avons refait la mesure avec `$bsonSize` dans un pipeline d'agrégation.
3. **Partie 6.** Le script proposé comptait les documents de chaque shard avec `$collStats`, qui compte aussi les orphelins laissés après un `sh.moveChunk` : nos preuves affichaient 100 000 documents au lieu de 60 000. Nous l'avons repéré en relisant les chiffres, puis mesuré la vraie répartition avec `$shardedDataDistribution` (`preuves/06-orphelins.txt`).
4. **Partie 7.** Le script de sécurité supposait que `getRoles` renvoie un simple tableau, ce qui n'est pas le cas dans mongosh 2.14.0 (`TypeError: b.getRoles(...).map is not a function`). Nous l'avons fait corriger et avons refait toute la preuve (`preuves/07-securite.txt`).